GeoLog AI

Pilot Readiness Checklist · 2026-05-03

Current status: READY FOR CONTROLLED FIELD TEST

Not yet ready for unobserved multi-tech pilot.

Why not pilot yet

The code is in good shape — state, recording, sync, and interruption handling have all been hardened through repeated audits. What's missing is the operational layer around it: real-device QA, network resilience, the export deliverable, and a way to debug failures 200 km from a desk. The items below are not code review findings; they are field-readiness gaps that code review cannot validate.

1. No real-device testing yet

Every interruption fix (visibilitychange, pagehide, mediaRecorder.onerror, MIME fallback) was verified by reasoning about specs. Real iOS Safari and real Android Chrome on actual phones during real phone calls are needed before unobserved use.

- iOS Safari 14 / 15 / 16 differences in MediaRecorder codec output
- Android Chrome behaviour when an incoming call notification appears (often does NOT fire visibilitychange)
- iOS Low Power Mode throttling background fetch on the upload step
- Bluetooth headset switching mid-recording
- Cellular handoff (LTE→5G→LTE) during a 30s upload

2. No backend load or failure-mode testing

Whisper latency, concurrent uploads, and Supabase rate limits have not been exercised under realistic conditions.

- Whisper takes 90s instead of 10s — frontend has no upload timeout, will wait forever
- Two techs uploading drafts on the same project simultaneously — never tested
- Supabase 5xx / rate-limit responses surface as generic 'Pipeline error'; no retry-vs-fail distinction

3. Open data-integrity gap (low frequency, real)

Backend confirm_sample (app.py:316–372) does a non-atomic check-then-insert. Two devices saving simultaneously at the same depth can both pass the pre-check before either inserts. Supabase rejects the second on the unique constraint (good), but the frontend will surface a confusing 'duplicate' error after the user thought their save was in flight.

- Add a UNIQUE constraint test
- Surface a clearer 'this exact sample already exists' message distinct from generic save errors

4. No PDF export

The product promise is 'Generate Borehole Log PDF' once samples exist. The button is wired-but-disabled. Pilot users will lose trust if a full day of logging produces no exportable artifact.

5. No offline / poor-network mode

Field sites have patchy LTE. The retry button helps for one failed upload, but if the tech logs five samples in a no-signal area, they will lose all five. The whole point of the app is that it works at the drill rig, not at the office.

- Offline draft queue (record + queue + flush when network returns)
- Local persistence of recordings until confirmed-saved server-side

6. No observability

If a tech reports 'the app didn't save my sample,' there is no way to investigate. No client-side error logging, no server-side request log surfacing, no support contact path. Pilot without this is operating blind.

- Client error logging (Sentry or equivalent)
- Surface request-id / sample-id in success and error toasts so a tech can quote them

Pilot-ready checklist

Ordered by importance. Items at the top must land before pilot; items at the bottom are already confirmed durable from code review.

✗ **OPEN**	Real-device QA: iOS Safari + Android Chrome on actual phones, real phone calls, real cellular
✗ **OPEN**	Add fetch timeouts (60s for draft, 30s for everything else)
✗ **OPEN**	Offline draft queue (record + queue + flush when network returns)
✗ **OPEN**	PDF export endpoint + wire the disabled button
✗ **OPEN**	Atomic confirm (Supabase RPC with constraint, or rely on the unique index and surface duplicate cleanly)
✗ **OPEN**	Client error logging (Sentry or equivalent)
✗ **OPEN**	Battery test: a tech doing 30 samples on a single charge with screen-on
✓ **DONE**	Code-level durability (state, recording, sync, interruption handling)
✓ **DONE**	Backend contract integrity
✓ **DONE**	Mobile UI affordances (touch targets, sunlight palette, one-handed cockpit)

Recommendation

Run the controlled field test first — one tech, you observing, ideally somewhere with bad signal on purpose. The findings from that session will tell you which of the seven open items above need to land before pilot.

Honest summary: the code is in good shape. What is missing is the operational layer around it — testing on hardware, network resilience, the export deliverable, and a way to debug failures when they happen 200 km from a desk.